

UNIT 9 · WEEKS 15–16

AI-native OS: agents and LLMs in the operating system

📅 Weeks 15–16 · dates to be confirmed </> Lab TD8 (solved + AI challenge) ~ 4 animations

MANUAL M9 · THEORY OF THE UNIT

AI-native OS: agents and LLMs in the operating system

Core reading for unit 9 (≈ 1.5 h). This unit brings the whole course together: an AI agent is one more process, using system calls, files, memory, threads and sockets, but whose decisions are made by a model that can be wrong. **The products cited change fast:** every claim carries its source and date (checked on 25/09/2026).

CONTENTS

- | | | | |
|----|----------------------------------|----|-------------------------------------|
| §1 | From chat to agent | §4 | Linux isolation primitives |
| §2 | What Windows and Linux announced | §5 | The model proposes, the OS disposes |
| §3 | Inference as an OS workload | | |

§1 From chat to agent

A language model (LLM) receives text and predicts the next text. An **agent** adds a loop: the model proposes an *action* —a tool call, such as `read_file("notas.txt")` —, a program runs it and the result goes back to the model. That way an assistant can organise files, change settings or type into an application.

For the operating system this is new for two reasons: the agent acts **on behalf of the user but is not the user**, and its decisions come from a statistical model that can be wrong or be manipulated (for instance, with instructions hidden in a document it reads: *prompt injection*). That is why the questions of this unit are operating-system questions: who is the agent?, what can it touch?, who authorises it?, how is it audited?

§2 What Windows and Linux announced

Windows

- **Agent workspace:** a separate space where the agent works in parallel with the user, with **its own account**; it accesses the known folders (Documents, Downloads, Desktop, Music, Pictures, Videos) only if the user enables it and consents. It is turned on in *Settings* → *System* → *AI*

components → *Experimental agentic features* (preview). Source: Microsoft Support, "Experimental agentic features".

- **Agent connectors:** *Model Context Protocol* (MCP) servers that connect the agent to applications and system tools, registered in the **On-Device Registry**; the first ones, for File Explorer and Settings (Insiders, December 2025). Sources: Microsoft Support; The Register, 08/12/2025.
- Declared principles: agent actions **observable and distinguishable** from the user's, user **authorisation** for data and actions, and **least privilege** (never more permissions than the user who started it).
- **Microsoft Execution Containers** (Build 2026, 2 June 2026): an execution layer with declarative policies for agents on Windows and WSL, with isolation per *process*, per *session* or in a hypervisor-backed *micro-VM*; in early preview. Source: Windows Developer Blog, "Windows platform security for AI agents", 02/06/2026.

Linux

- There is no single "system agent": the **kernel provides primitives** (§4) and distributions and applications combine them.
- **Canonical (Ubuntu)** announced in 2026 AI features focused on local inference with open models (*inference snaps*) and agents confined with snap mechanisms; the first previews are expected in Ubuntu 26.10. There will be no global AI switch: it is turned off by removing the snaps. Sources: OMG! Ubuntu, 04/2026; Phoronix.
- For AI accelerators, Linux has the **accele** subsystem (Intel and AMD NPU drivers). In the kernel community, since December 2025 the documentation asks to declare help from AI tools with the **Assisted-by** tag; only a person can add the **Signed-off-by** sign-off.

Practice this → animation A9.1 · Architecture of an agentic OS

§3 Inference as an OS workload

For the operating system, running a model is a program with a very particular profile:

- **Huge, read-only weights.** They are loaded with **mmap** (unit 6): not copied but read from the page cache when touched (page faults), and several processes share the same physical pages. The TD8 micro-LLM maps its ≈ 0.7 MB of weights that way.
- **The KV cache.** For every token already processed, the model keeps the keys and values of every layer so as not to recompute them: memory that grows with the length of the conversation (in TD8, $2 \times \text{layers} \times \text{context} \times 64 \times 4$ bytes).
- **Two phases.** *Prefill* processes the whole request at once; *decode* generates one token per step, each reading all the weights.
- **Accelerators.** GPUs and NPUs are handled as devices: drivers, their own memory and scheduling of access among processes.

Practice this → animation A9.2 · Inside the micro-LLM

§4 Linux isolation primitives

PRIMITIVE	WHAT IT LIMITS	NEEDS ROOT
Users and permissions	Files according to owner, group and mode	To create users
Landlock (since 5.13)	Which file-system paths the process may open; TCP ports (ABI 4); signals and abstract sockets outside its scope (ABI 6)	No : the process restricts itself
seccomp	Which system calls it may make	No (with <code>no_new_privs</code>)
Namespaces	What it sees: processes, mounts, network, users (the basis of containers)	No, with user namespaces
cgroups	How much it consumes: CPU, memory, I/O	Usually yes (or delegated by systemd)
<code>setrlimit</code>	Per-process limits: CPU time, file size, descriptors	No, to lower them

TD8 uses Landlock, `setrlimit` and `alarm` in `agent_exec`: after `landlock_restrict_self` the process can no longer leave the workspace, even if its own code tries. The restriction is inherited and cannot be undone.

§5 The model proposes, the OS disposes

The central lesson of the unit: **the model output is untrusted text**. TD8's agentic OS treats it like any external input:

1. **Strict parser**: only `name("arg", "arg")` from a closed catalogue; everything else is rejected.
 2. **Policy**: relative paths inside the workspace, without `..` and resolving symbolic links (`realpath`).
 3. User **consent** for anything destructive.
 4. **Kernel sandbox** (Landlock) as the last line: if the policy fails or is disabled, the kernel still denies access.
 5. **Auditing**: every decision goes to `audit.log` with the client's uid and pid (`SO_PEERCRED`).
- It is the principle of *defence in depth*: no layer trusts that the previous one did not fail.

Practice this → animation A9.4 · The model proposes, the OS disposes, and TD8